

EasyTrainer 使用教程

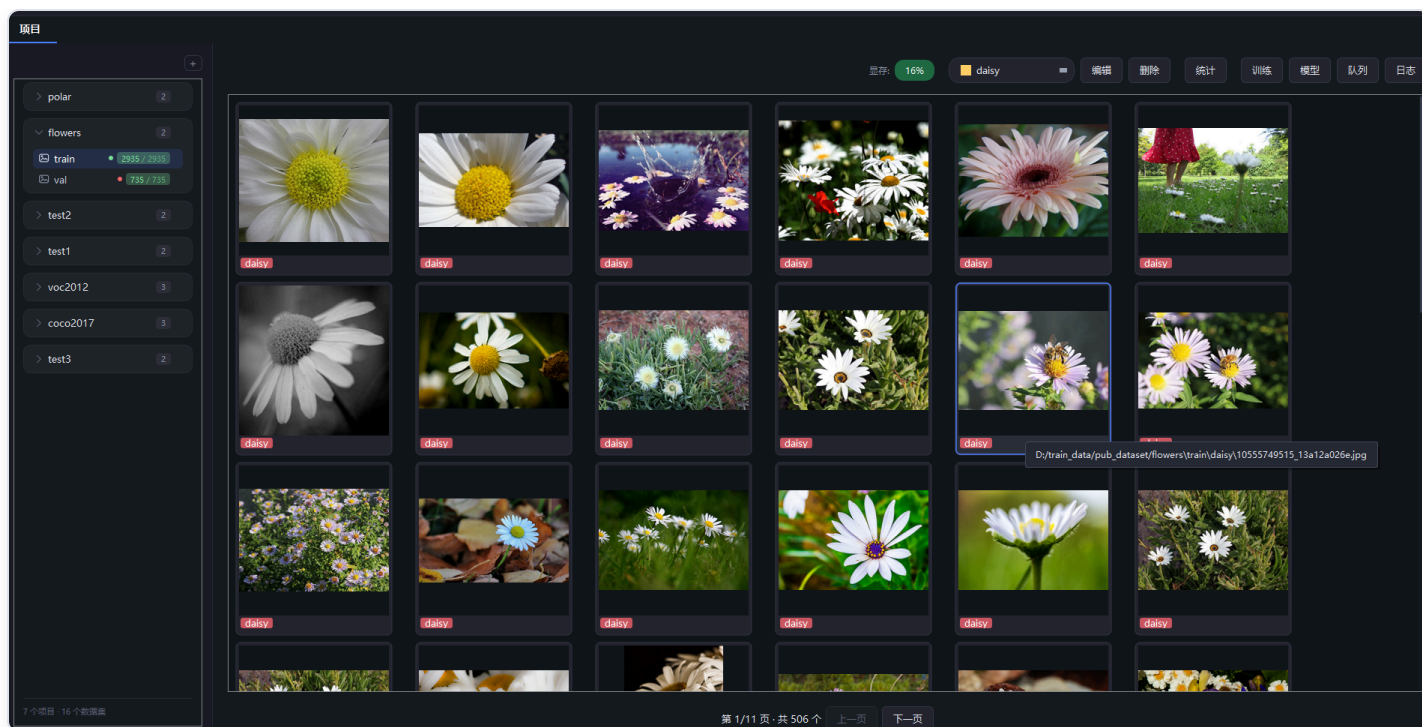
目录

1. 软件概览
2. 项目与数据集管理
3. 标签管理
4. 图像标注
5. 训练与训练队列
6. 模型管理
7. 模型测试与评估报告
8. OCR (字符检测 + 字符识别)
9. 日志

1. 软件概览

主界面采用三栏布局：

- **左侧项目树**：项目与数据集层级展示，数据集节点带标注进度（如 126/128）；名称前的圆点表示该数据集**是否已加载进内存**（绿=已加载，红=未加载）
- **中间图像区**：缩略图分页浏览，底部居中一组翻页控件（两个箭头夹住页码，当前页数字化亮），悬停页码可看当前筛选下的总数与单位
- **顶部工具栏**：显示比例、标注筛选，以及编辑、删除、导入、导出、属性、训练、模型、日志等按钮
- **界面语言**：工具栏最右侧的下拉框，可选 **中文、English、繁體中文、日本語、한국어、Deutsch、Español、Français、Tiếng Việt** 九种。主界面当场就变，已经打开的窗口（日志、训练队列）也一起刷新；还没打开的对话框，下次打开自然就是新语言。选择会记住，下次启动仍是该语言。下拉框里列出的是各语言自己的写法，跟当前界面语言无关，所以任何语言下都能认出该选哪一项。
译文按语言放在 `i18n/<语言代码>.ts`（`.qm` 由它编译出来），改过界面文案后跑一次 `python builder/translations.py` 重新抽取并编译全部语言。新抽出的条目会标成未完成，编译时跳过并回退中文，所以漏译最多是那一句显示中文，不会让界面空掉

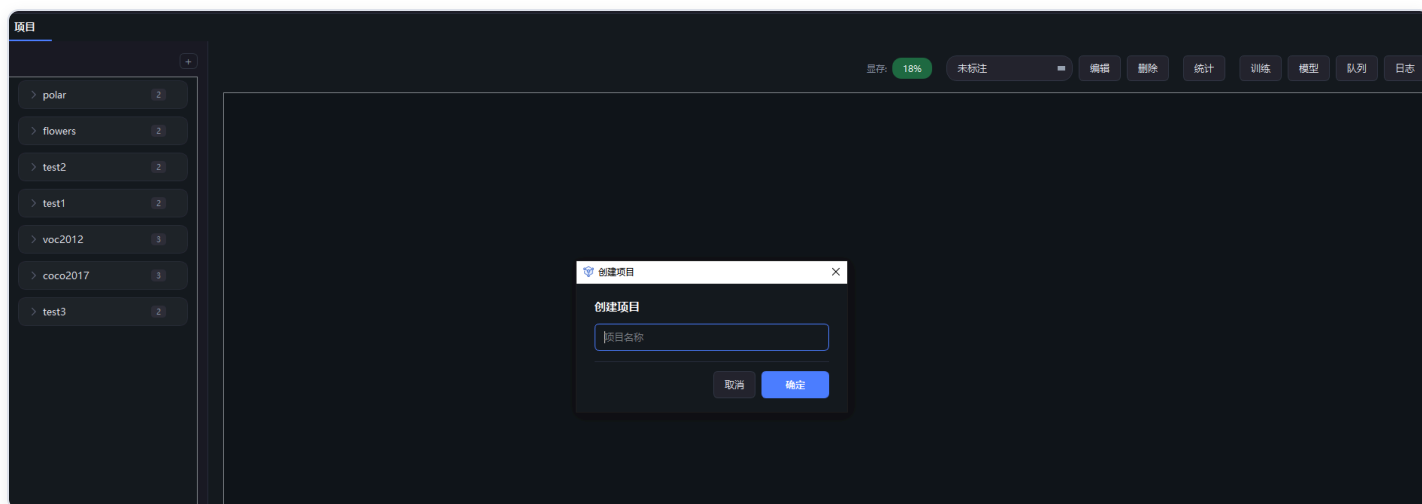


EasyTrainer 主界面

2. 项目与数据集管理

2.1 添加项目

点击左侧 **+ 添加项目** 按钮，输入项目名称后确定。项目是数据集的容器，所有数据集必须归属于某个项目。



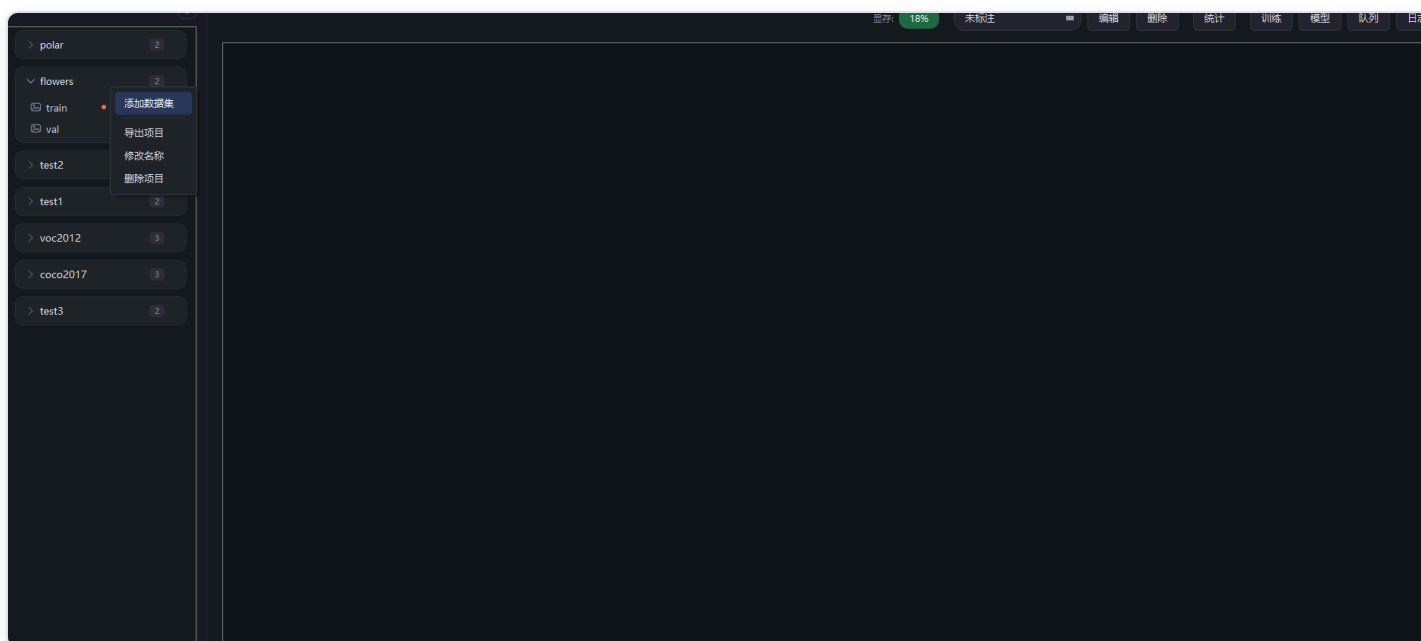
输入项目名称

2.2 添加 / 重命名 / 删除数据集

右键点击**项目节点**弹出菜单：添加数据集 / 导出项目 / 修改名称 / 删除项目。

- **添加数据集**：在项目下新建一个数据集
- **导出项目**：把整个项目的数据集与标注导出为指定格式
- **修改名称**：重命名项目
- **删除项目**：连同其下数据集、标注、训练记录一并清除

右键点击**数据集节点**则是对该数据集的操作：导入 / 导出 / 移动 / 修改 / 删除。**双击数据集节点**即载入：重新扫描该数据集的图像与标注并重建缓存，随后即可在右侧浏览。已载入（行内有状态圆点）的数据集再次双击不会重复扫描。



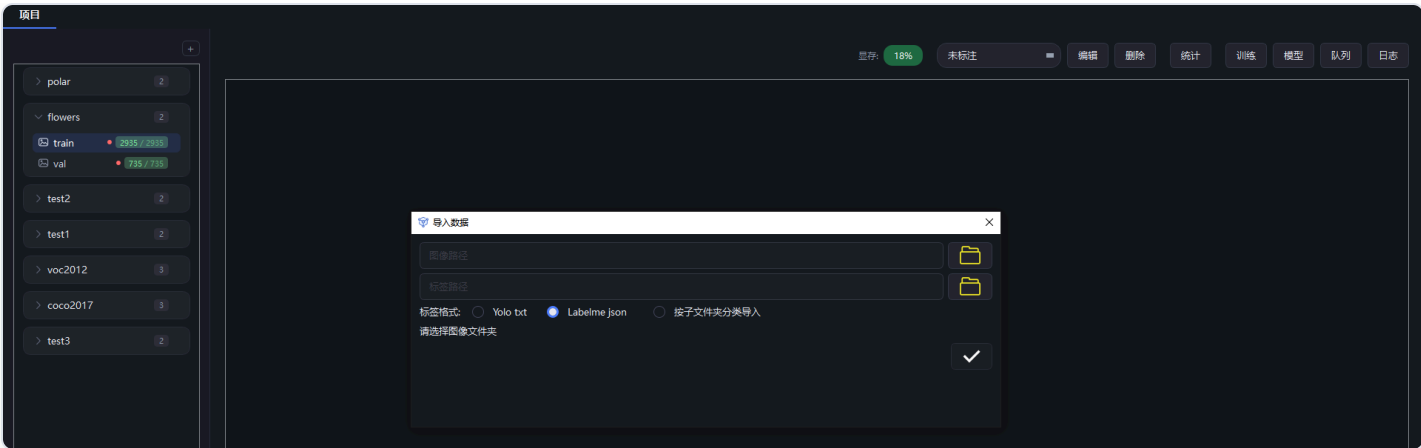
项目右键菜单

2.3 导入数据

选中数据集后，点击顶部 **导入** 按钮，弹出"导入数据"对话框：

- **图像路径**：选择图片所在文件夹
- **标签路径**：选择标注文件所在文件夹
- **标签格式**：Yolo txt / Labelme json / 按子文件夹分类导入（OCR 用 Labelme json）

分类数据集选择"按子文件夹分类导入"：每个子文件夹名作为一个类别，**无需标注文件**；OCR 用 Labelme json，文本框的标签固定为"文本"，导入后自动标成字符检测数据集。



导入数据对话框

2.4 数据集属性与标签分布

选中数据集后点击顶部 **属性** 按钮，可查看图像路径、标签路径，以及**标签分布柱状图**（每个类别的标注框数量），用于快速发现类别不平衡问题。



数据集属性与标签分布

3. 标签管理

3.1 添加或导入标签

在标注界面右侧标签列表上方点击 **添加标签**：

- **标签名称**：多个标签用逗号分隔，可一次添加多个
- **导入**：从已有数据集一键导入全部标签
- **颜色**：从预设色板选取，或点击"自定义"挑选颜色



添加或导入标签

3.2 批量修改类别

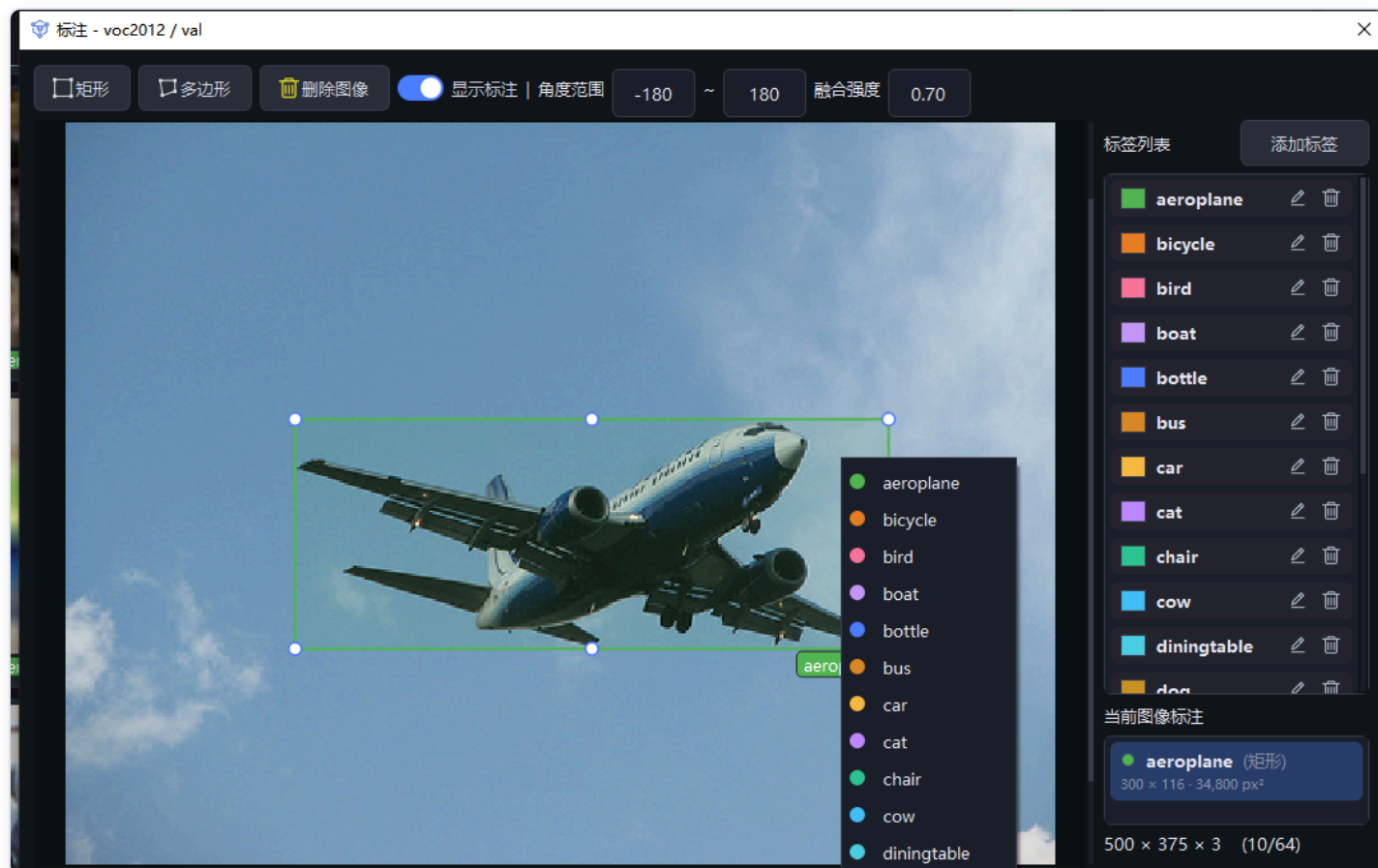
选中数据集后，在**图像区缩略图上右键**选择"批量修改类别"，输入原类别与新类别，即可把整个数据集中该类的标注全部改名：



类别批量修改弹窗

3.3 修改单个标注的类别

进入标注界面后，在标注框上点击即可弹出类别下拉菜单，选中新类别即完成修改；右侧标签列表中的标签也可重命名、改色、删除。



在标注框上直接修改类别

4. 图像标注

4.1 进入标注界面

在左侧树选中具体数据集后，右键缩略图选择“标注”，或直接双击缩略图进入。

4.2 矩形、多边形与文本标注

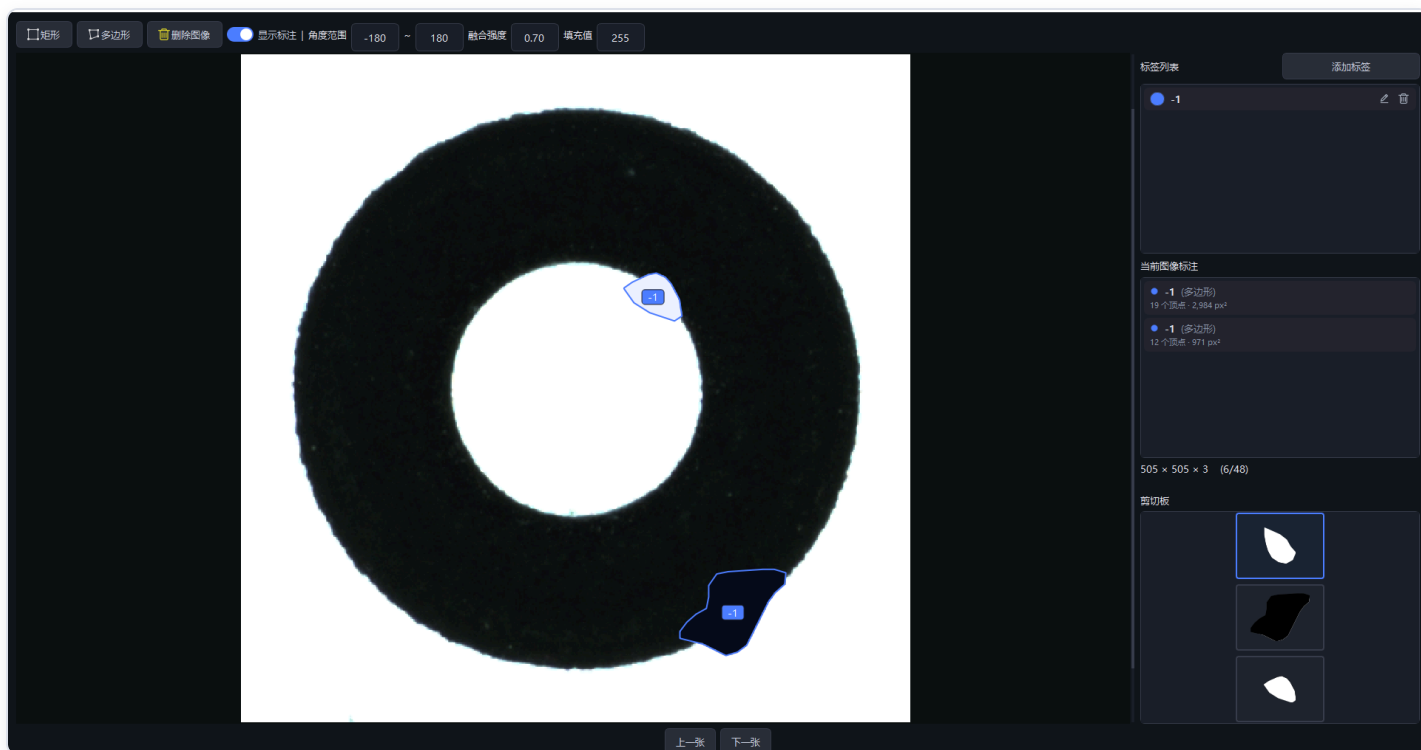
标注界面顶部工具栏提供三种绘制工具：

- **矩形**：拖拽鼠标画矩形框
- **多边形**：依次点击顶点，双击或回车结束（分割任务用）

- **文本**：先打开顶部“文本标注”开关再拖拽画框，松开即弹出输入框录入文字，双击可改（OCR 任务用）。文本框一律用**矩形**画，倾斜文字也用水平框盖住即可 —— 多边形的贴合轮廓在训练时会被按水平外接框处理，画了没有额外收益
- **删除图像**：删除当前这张图

右侧**标签列表**选择类别；**显示标注**开关可随时隐藏所有标注轮廓（框线、标签、顶点手柄）查看原图；尚未落地的粘贴内容只藏轮廓、图案照常显示，所以关掉它正好可以预览“贴上去之后是什么效果”。右下方“**标注信息**”面板列出本图所有标注对象的形状（矩形/多边形）、顶点数与面积；其下方**图像信息**显示当前图的宽 × 高 × 通道数（如 505×505×3）与序号。

“**图像信息**”下方为**剪切板区**：右键复制多边形后，被复制区域的缩略图（120 × 90）会加入这里，最多 9 张、纵向排列、可上下滚动，**最新一张默认选中并带蓝色边框**；右键缩略图可**删除该张**或**清空全部**。



矩形 / 多边形标注、标签列表与右下角剪切板

4.3 标注对象复制（数据增强）

右键**多边形**标注有两个操作：

- **复制**：把该多边形区域的像素存为模板，同时在右下角剪切板生成缩略图
- **填充**：把区域内的像素改成“填充值”三个输入框对应的 RGB 颜色（每通道 0 ~ 255，默认 255），常用于把缺陷区域抹成背景色

左键点选剪切板里的某张缩略图后，把鼠标移到画布上，会有一条**多边形虚线轮廓**跟着鼠标走（形状是旋转后的真实轮廓，已套用本次摇出的角度），据此可以先看清这一贴大概落在哪、朝向如何；按 **Esc** 收掉虚

线、回到普通鼠标模式，再右键选“粘贴”即可落地。

在图像空白处右键选择**粘贴**，即可把**剪切板中当前选中的**那张缩略图 贴到鼠标位置（同图或跨图均可）：

- **角度范围**：粘贴时随机旋转的角度区间（默认 -180 ~ 180）； 虚线预览用的就是这个区间里摇出的角度，所见即所得
- **融合强度**：0 ~ 1，默认 0.70。粘贴时自动做亮度补偿和边缘羽化，让粘贴区域与背景自然过渡；设为 0 则等同直接硬贴
- **填充值**：R - G - B 三个输入框（每通道 0 ~ 255，默认 255），配合右键“填充”把区域抹成该颜色
- **Ctrl + Z**：撤销最近一次粘贴或填充（恢复像素并移除随之新增的标注），可连续撤销



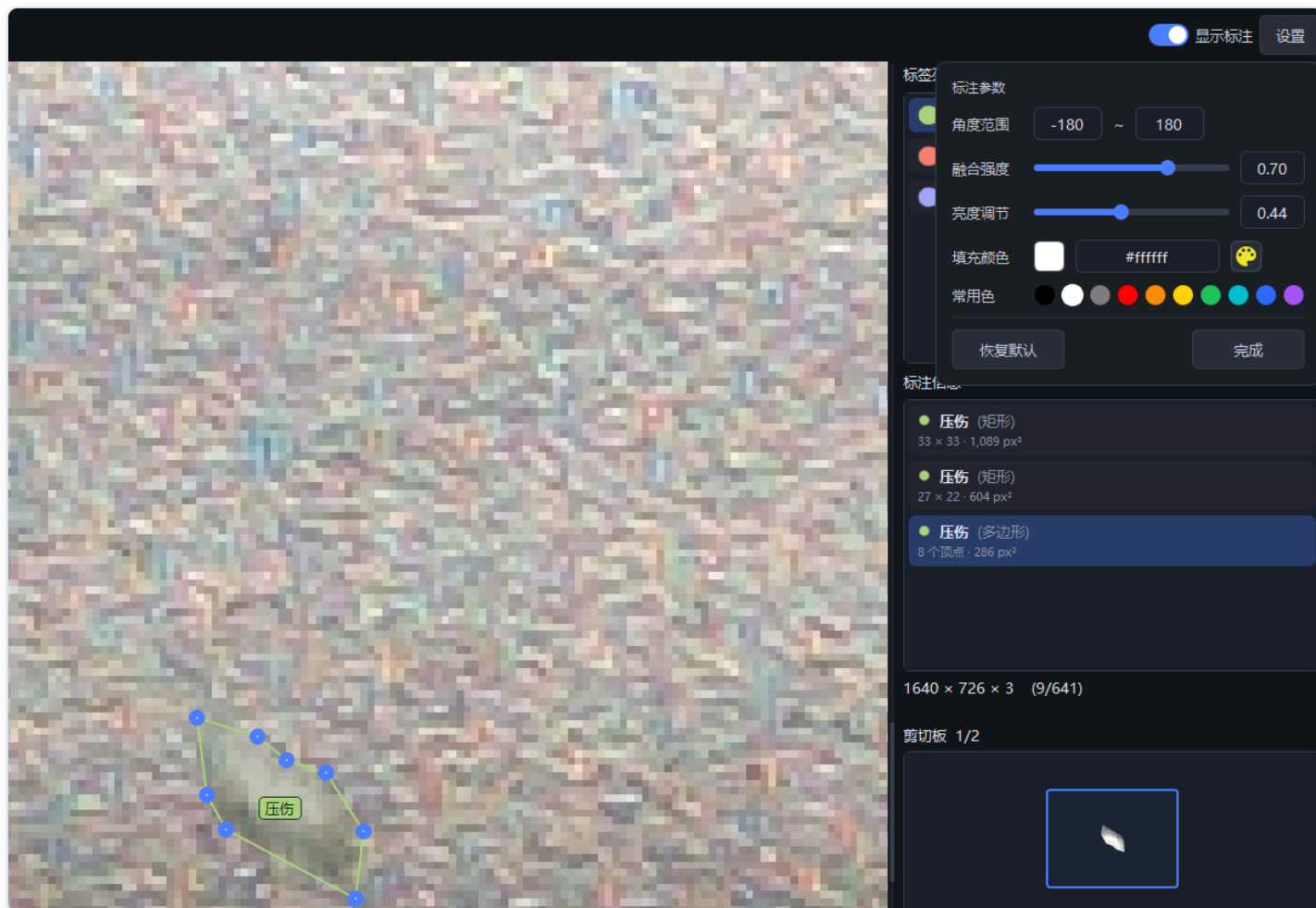
右键复制标注对象 → 剪切板缩略图，工具栏设置角度范围 / 融合强度 / 填充值

4.4 多边形区域亮度调节

在画布上点选一个**多边形**标注，工具条“设置”里的 **亮度调节**即可改该多边形**框内像素**的明暗：拖动滑块（或直接输入数值）整体提亮或压暗，图像上实时预览。

- **范围与含义**：0 ~ 1，默认 **0.50 = 原样**；越往 1 越亮（1.00 约为两倍亮度），越往 0 越暗（0 为全黑）
- **作用对象**：只对**选中的多边形**生效；矩形或没选中任何标注时给提示、不动像素。点另一个多边形时滑块回到它自己的值（没调过的都是 0.50），上一个多边形的亮度不会跟着带过去

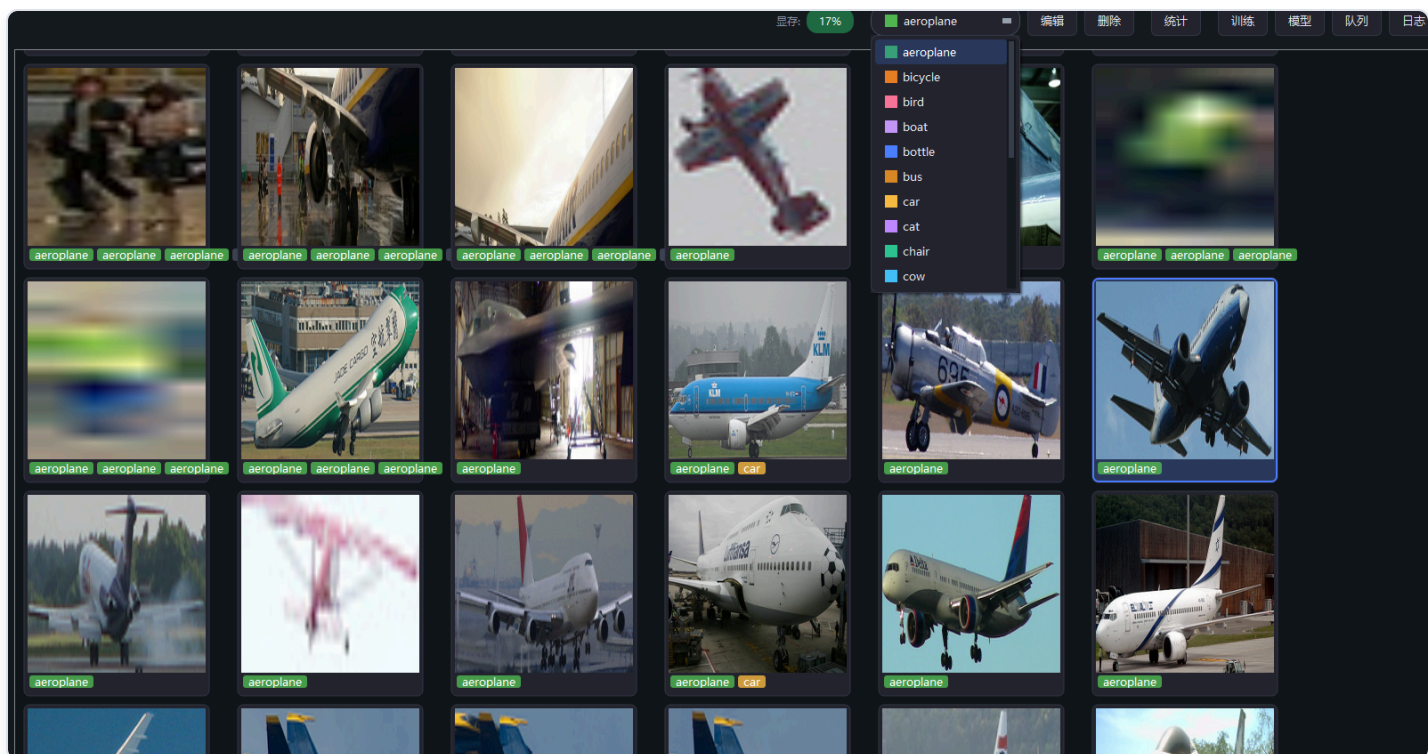
- **什么时候写盘**：拖动过程只改内存、实时预览，按 **A** / **D** 切图 或 **Ctrl + S** 才写进图像文件
- **撤销**：一次调节只记一步，**Ctrl + Z** 一下即可回到调节前的像素（同一次里来回拖多少次也只算一步）



选中多边形后用“亮度调节”改框内像素明暗，图像上实时预览

4.5 按标注类别筛选

首页顶部工具栏的类别下拉框，选择某类别后图像区只显示含该类别的图，便于集中检查、补标某一类。



按类别筛选缩略图

5. 训练与训练队列

5.1 训练参数配置

点击首页或模型界面的**训练**按钮，弹出训练参数配置：

- **任务类型**：检测 / 分割 / 分类 / 异常检测 / OCR（按数据集格式自动联动）
- **训练集 / 验证集**：多选数据集，可跨项目组合
- **设备**：自动列出可用 GPU（如 NVIDIA GeForce RTX 4060 Ti）
- **训练超参**：轮次、优化器（检测/分割=adamw、分类=sgd）、早停（连续 N 轮无提升则提前结束，0 为关闭）、学习率（训练中自动衰减）、批次、图像尺寸（32 的倍数）、梯度累积（显存不足时等效放大批次）、线程数
- **输出路径**：留空则自动按时间生成目录，也可点"选择路径"指定

训练

×

训练

检测

模型与数据

任务类型

检测

网络

nano

训练集

验证集

设备

NVIDIA GeForce RTX 4060 Ti (8 GB)

训练超参

轮次

100

优化器

adamw

早停

20

学习率

0.0001

连续无提升则提前结束，0 为关闭

初始学习率，训练中自动衰减

批次

4

图像尺寸

640

32 的倍数

梯度累积

4

线程数

4

显存不足时调大，等效批次 $\times N$

输出

输出路径

留空则自动按时间生成目录

选择路径

请选择训练集与验证集

×

加入队列

开始训练

训练参数配置对话框

5.2 训练队列

配置好后除"开始训练"外，还可以点**加入队列**把任务先挂起来，继续配置下一个。点击首页 **训练队列**（或训练窗口入口）打开队列窗口：

- 队列按顺序依次执行，**支持检测 / 分割 / 分类 / 异常检测 / OCR 任务混排**，自动逐项切换
- **上移 / 下移 / 移除**：调整执行顺序或删除任务
- **编辑**：修改选中任务的参数

- **清理已完成**：移除已跑完的记录
- **开始队列**：从头依次执行队列中的任务

队列适合晚上挂机：把多个数据集的任务一次配好，软件会自动跑完全部并分别保存模型。

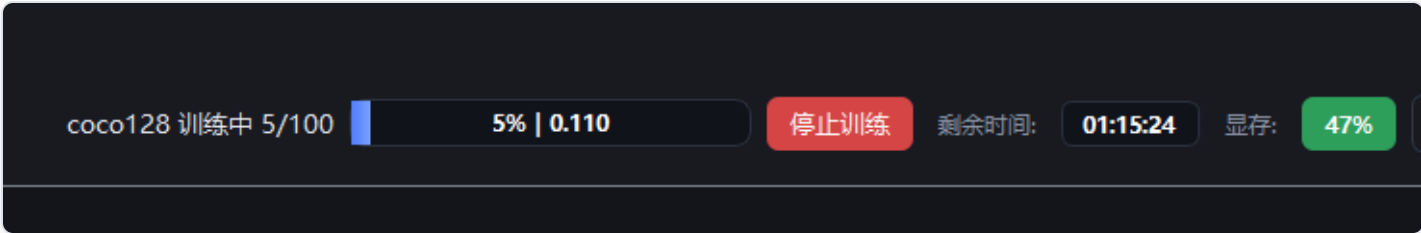


训练队列窗口

5.3 训练进度监控

点击"开始训练"后回到首页，顶部出现进度条与控制条：

- **任务名 + 进度**：如 "coco128 训练中 5/100"，进度条内显示当前轮 loss
- **停止训练**：随时终止（已完成的部分会保存）
- **剩余时间**：按已用时长外推估算
- **显存**：实时显示 GPU 显存占用率

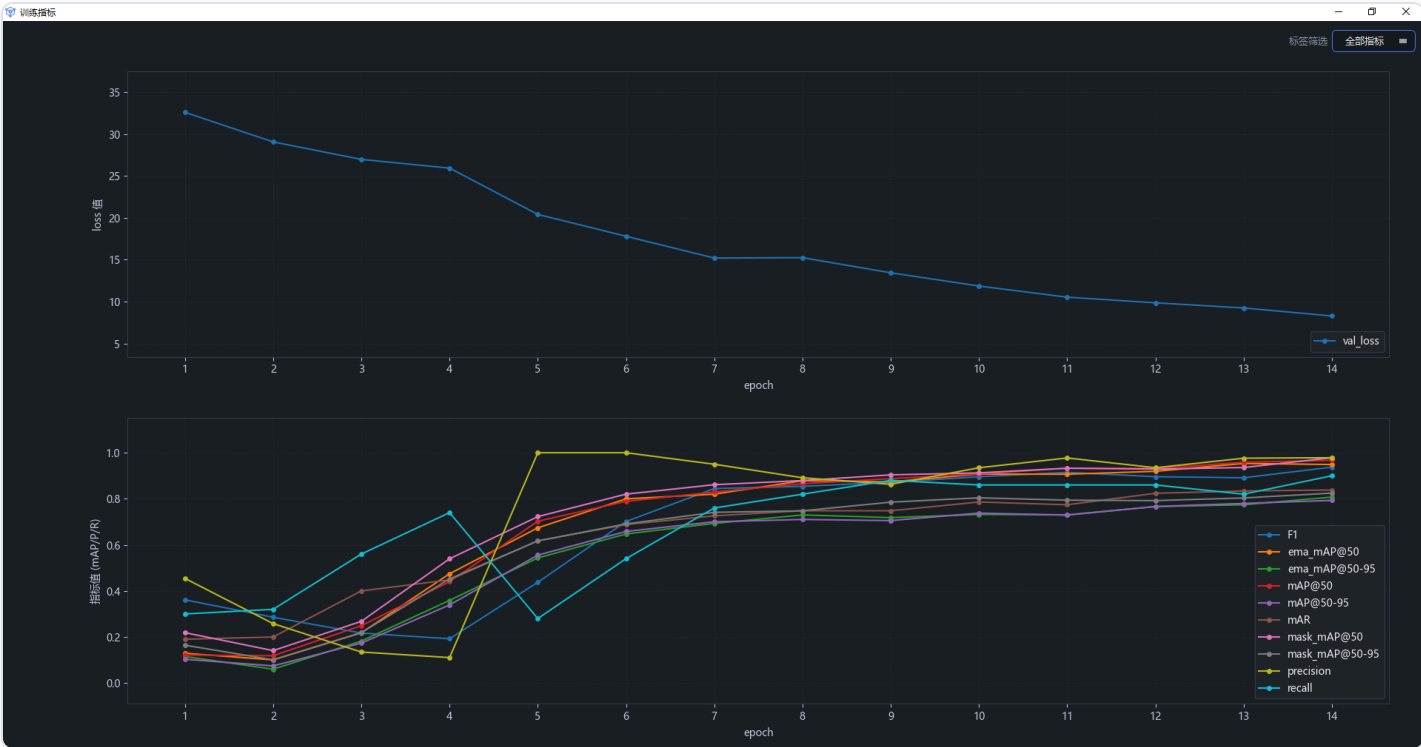


训练进度条与状态栏

5.4 训练指标查看

在模型管理窗口选中某条训练记录，点击右侧详情面板的**查看完整指标**，弹出训练曲线窗口：

- 上半图：**val loss** 曲线
- 下半图：mAP@50-95 / mAP@50 / AR / F1 / precision / recall 等多条曲线（分割任务还会显示 mask mAP 曲线）
- 右上角"标签筛选"下拉框可切换查看全部指标或某个类别的曲线



训练指标曲线（val loss + 各项精度）

6. 模型管理

点击首页 **模型** 按钮打开模型管理窗口，左侧为训练记录列表， 右侧为固定详情面板：

6.1 筛选与列表

- **搜索框**：按项目 / 数据集 / 标签关键字过滤
- **任务 / 状态下拉**：只看检测 / 分割 / 分类 / 异常检测 / 字符检测 / 字符识别，或训练中 / 已完成 / 失败 / 已停止
- **仅看每个数据集最佳**：同一数据集只保留精度最高的一条
- **列表列**：任务、数据集-标签、精度、训练时间、耗时、图像尺寸、操作
- **精度列带色条**：绿 ≥ 0.8 、黄 $0.5 \sim 0.8$ 、红 < 0.5 ；失败/已停止的记录在精度列显示状态文字，悬停可看失败原因
- **行内按钮**：**测试**（打开模型测试）、**导出**、**删除**（同时清理对应训练记录）

点击**导出**选择目录后，会生成「项目_时间戳」文件夹，里面包含：

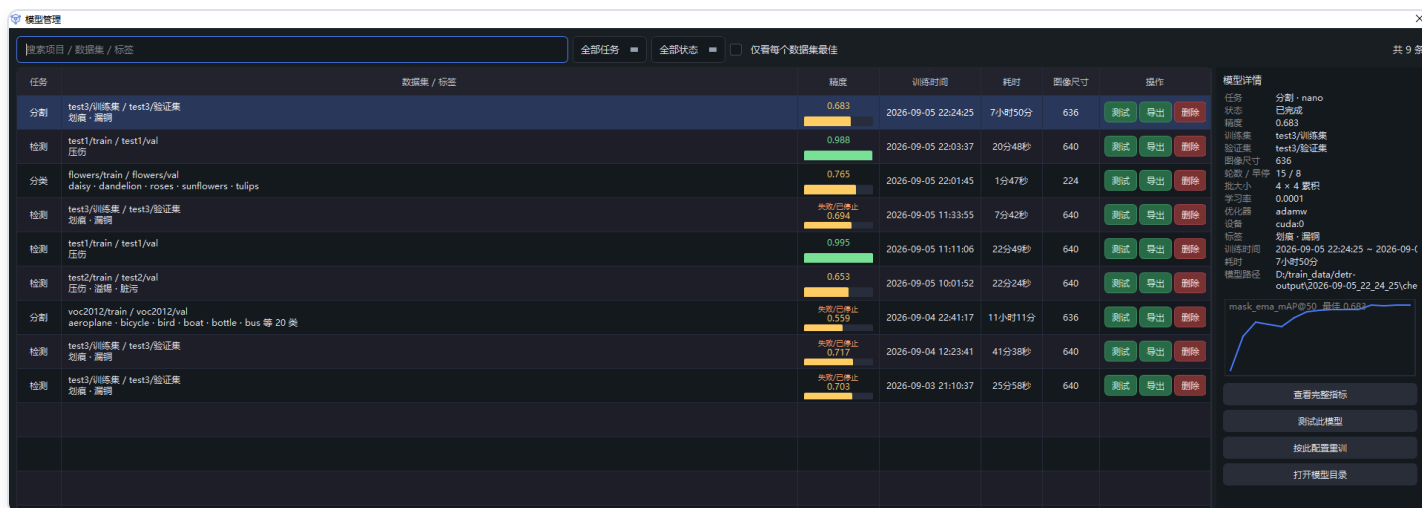
- **模型 .onnx**：检测/分割由 RF-DETR 导出，分类与 OCR 由 torch 导出，命名「项目_任务_图像尺寸_模型规模.onnx」
- **classes.txt**：类别 id 与名称对照（OCR 识别段是 `vocab.txt`，一行字符即词表）
- **评估报告 PDF**：自动在验证集上跑一遍评估，复用测试报告的模板，命名「..._评估报告.pdf」
- **examples/**：C++ / C# / Python 三种语言调用 ONNX 做检测、分割、分类的示例代码（OCR 目前只有 Python 版）

导出过程有进度提示；ONNX 转换需要 `onnx` / `onnxsim`，未安装会在弹窗里提示。分类任务不生成评估报告（检测/分割的漏检误检报告不适用），OCR 的字符识别同理（结论只有 CER）。

6.2 右侧详情面板

点击任意一行，右侧显示该记录的完整参数：任务-规模、状态、精度、训练/验证集、图像尺寸、轮数/早停、批大小、学习率、优化器、设备、标签、训练时间、耗时、模型路径，上方还有**精度曲线缩略图**。面板底部四个按钮：

- **查看完整指标**：打开训练曲线窗口（见 5.4 节）
- **测试此模型**：用该模型打开测试对话框（见 7.1 节）
- **按此配置重训**：带出该记录的全部训练参数重新开一次训练
- **打开模型目录**：在资源管理器中定位权重文件



模型管理：筛选条 + 训练记录列表 + 右侧详情面板

7. 模型测试与评估报告

7.1 测试参数设置

在模型管理窗口点击行内**测试**按钮（或详情面板的“测试此模型”），弹出模型测试对话框。顶部显示模型信息（任务类型、权重文件、训练精度、输入尺寸、规模），下方配置：

- **数据**：跨项目所有数据集可选
- **设备**：默认第一个可用 GPU
- **置信度**：低于该分数的预测直接丢弃
- **IoU 阈值**：预测框与标注框重合度达标才算正确检出
- **输出标签文件**：勾选后把预测框写成 labelme json，便于人工复核
- **OCR**：字符检测分“推理 / 评估”两条路 —— 推理把结果写成标注 json，评估统计 P/R；字符识别不输出标注文件，只报告字条识别率

数据集已标注时自动进入**评估模式**：统计检出率 / 漏检 / 误检；字符检测若选推理模式，则无视标注、直接把预测框写成 json。

模型测试

×

模型测试

分割

checkpoint_best_ema.pth

mask mAP50 0.683 · 输入 636 · 规模 nano · 训练 2026-09-05 22:24

数据与设备

数据

test3/训练集

设备

NVIDIA GeForce RTX 4060 Ti (8 GB)

测试参数

置信度

0.5

低于该分数的预测直接丢弃

IoU 阈值

0.5

与标注框重合度达标才算正确检出

输出标签文件

☐ 把预测框写成 labelme json, 便于人工复核

1 个数据集 · 993 张图 · 已标注, 评估模式: 统计检出率 / 漏检 / 误检

×

开始测试

模型测试参数配置

7.2 测试结果分析

测试完成后弹出"测试结果分析"窗口, 分两个维度统计 (全部用直白的 "正确检出 / 漏检 / 误检"表述):

- **图像维度** (按张统计, 检出 1 个即算检出): 测试张数、检出图像及检出率、未检出图像、有误检图像
- **标注维度** (按标注框统计): 正确检出、漏检、误检、准确率
- **按类别表格**: 每类的标注数 / 正确检出 / 漏检 / 误检 / 检出率 / 准确率
- **字符识别**: 不按框统计, 报告字条总数、CER (字符错误率) 与全对条数
- **底部结论**: 一句话点出整体偏漏检还是偏误检、拖累最大的类别



测试结果分析窗口（图像维度 + 标注维度 + 按类别明细）

7.3 导出 PDF 报告

点击右下角 **导出 PDF 报告**，后台线程生成评估报告，内容包含：

- 总览页：指标汇总表 + 标注分布图
- 漏检样本页、误检样本页：按“每类抽取 N 张”（左下角可调）展示缩略图，图上用红框标出漏检 / 误检位置，并注明抽取比例
- 建议页：针对各类错误给出改进方向

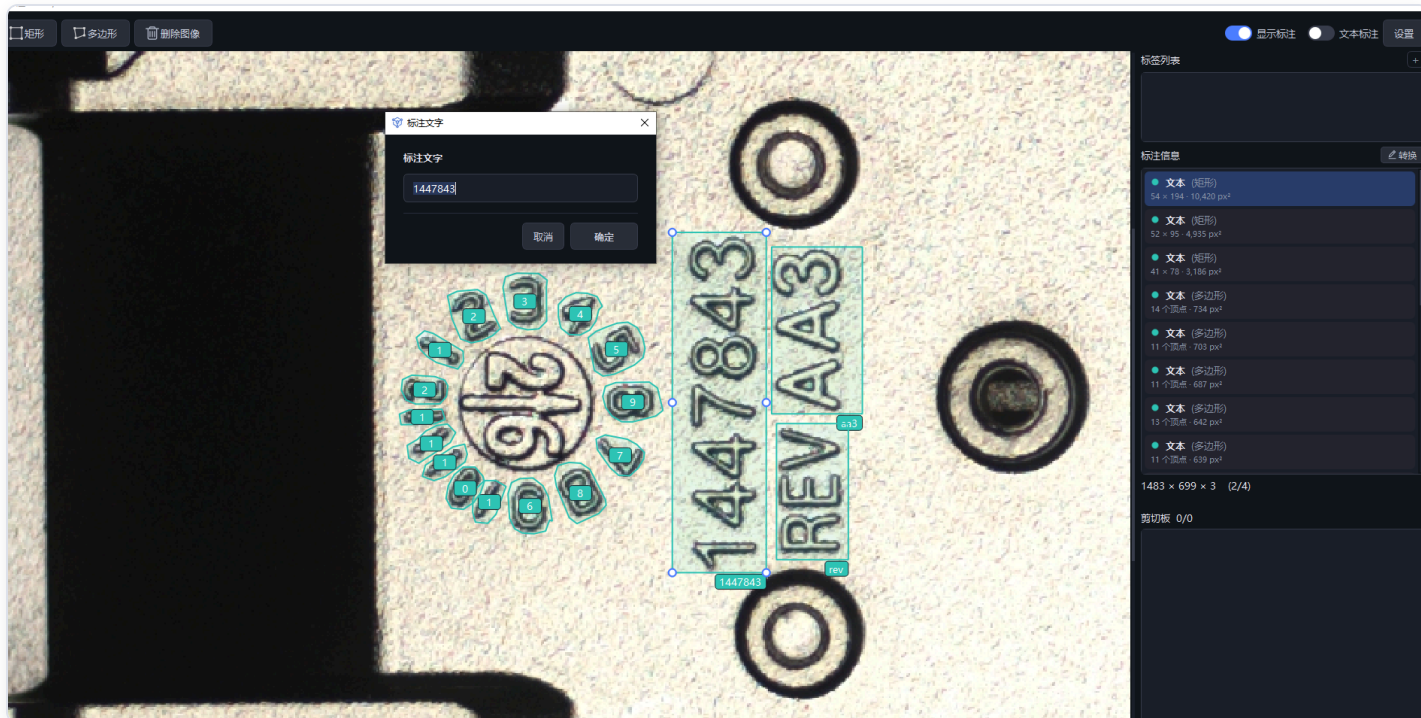
抽样只发生在出报告这一步，明细数据仍是全量，后续做难例挖掘不会丢数据。

8. OCR (字符检测 + 字符识别)

OCR 与其它任务不同：它由**两个模型**串联完成——字符检测负责找出文字框，字符识别负责读出框里的字。训练、测试、导出都是两段分开进行的，模型管理里会留下两条记录。

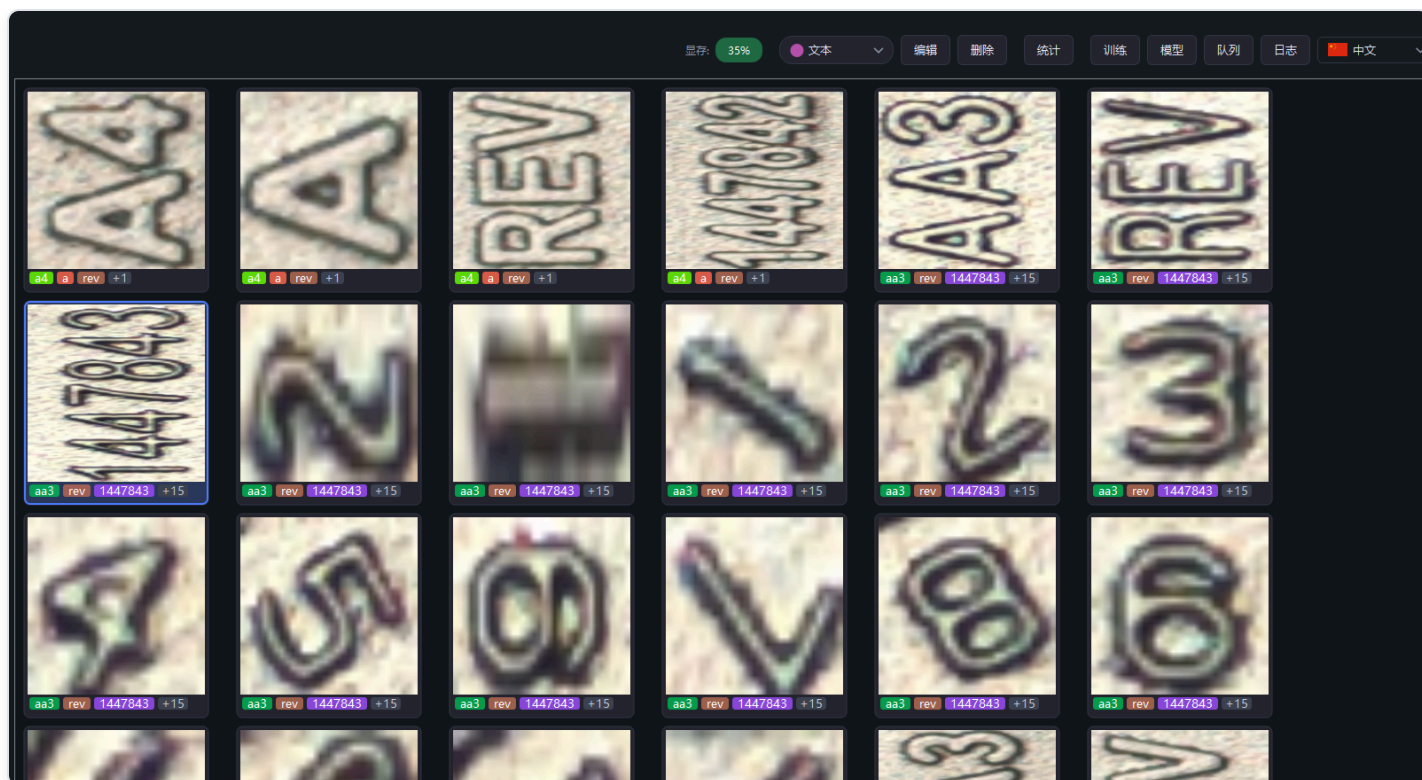
8.1 标注文本

在标注界面打开顶部的**文本标注**开关，类别会自动固定为保留标签“文本”（不必先建标签），像画矩形一样框住文字，松开鼠标即弹出输入框录入内容；双击已有的文本框可以随时修改。**文本框一律用矩形画**：倾斜的文字也用水平框盖住即可，训练时多边形会被按水平外接框处理，描着轮廓画没有额外收益。带文本框的 labelme json 在导入时会自动把数据集标成**字符检测数据集**，后续只有这类数据集会出现在 OCR 的训练集下拉里。



打开“文本标注”开关后拖框，松开即弹出文字输入框；右侧“标注信息”列出每个文本框的形状、顶点数与面积

标注完成后回到数据集视图，OCR 数据集里每个文本框会**单独裁成一张缩略图**，标签条上显示的是框里录入的**文字内容**（超过 3 条折叠成 +N），而不是类别名——逐张翻过去就能核对内容录得对不对。上方的类别筛选里只有保留标签“文本”一项。



OCR 数据集：每个文本框单独成图，标签条显示所录文字内容（超出折叠成 +N）

8.2 训练

任务类型选 **OCR**。图像尺寸对应检测段的输入（默认 1024，需为 32 的倍数），识别段固定 32×128，不受这一项影响。点“开始训练”后会**自动拆成两项排队**：先字符检测、再字符识别；前一项跑完，等显存回落到开训前的水平再起下一项，这期间进度区显示“等待显存释放·下一项:字符识别”，中间不会有空白。

训练

×

训练

字符检测

模型与数据

任务类型

字符检测

↓

型号

nano

↓

训练集

• train ×

↓

验证集

• train ×

↓

设备

GeForce RTX 4060 Ti (8 GB)

↓

架构

Transformer

↓

训练超参

轮次

50

优化器

adamw

↓

早停

20

学习率

0.0001

连续无提升则提前结束，0 为关闭

初始学习率，训练中自动衰减

批次

4

图像尺寸

1024

32 的倍数

梯度累积

4

线程数

4

显存不足时调大，等效批次 × N

输出

输出路径

D:/train_data/ocr_output

选择路径

ℹ 训练集 1 个 · 验证集 1 个 · 共 1624 张图 · 已标注, 可直接训练

×

加入队列

开始训练

OCR 的训练参数（图中标识为“字符检测”段）；型号即档位 nano ~ large，架构一栏由算法自动决定、置灰不可改

识别段的词表直接取标注里出现过的字符，所以标注文本要覆盖现场实际会遇到的字符集。

8.3 测试：字符检测两条路，字符识别一条路

- **字符检测 · 推理**：不比对标注，直接把预测框（连同识别出的文字）写成 labelme json。写出的 json 带 OCR 标志位，拖回软件会被认成字符检测数据集，可用来核对结果或补标。
- **字符检测 · 评估**：数据集已标注时自动进入，统计正确检出 / 漏检 / 误检与按类 P/R，和检测任务一致。
- **字符识别**：拿标注框裁出的字条来测，报告字条总数、CER（字符错误率）与全对条数，不输出标注文件。

8.4 导出

两段各导一份 ONNX： `项目_字符检测_尺寸_规模.onnx` 与 `项目_字符识别_尺寸_规模.onnx`，识别段额外带 `vocab.txt` 词表。字符检测照常生成评估报告；字符识别只有 CER 一个结论，不生成报告。调用示例见导出目录下的 `examples/python/ocr.py`（检测段输出的 logits 没过 sigmoid，后处理要自己做，示例里都写好了）。

9. 日志

点击首页顶部 **日志** 按钮，打开日志窗口，实时显示：

- 软件启动 / 退出
- 项目 / 数据集创建、重命名、删除、导入 / 导出
- 标签创建、重命名、删除
- 训练开始、训练中（[train] 前缀转发子进程输出）、指标更新、模型保存、手动停止
- 测试启动、测试中（[test] 前缀）、完成、失败

日志

清空

[2026-08-19 14:43:57]	[train]	48	1.0000	1.0000	1.0000	1.0000	1.0000
[2026-08-19 14:43:57]	[train]	49	0.7715	0.8000	0.8889	0.8000	1.0000
[2026-08-19 14:43:57]	[train]	50	0.2633	0.4545	0.3077	1.0000	0.1818
[2026-08-19 14:43:57]	[train]	51	0.2030	0.5750	0.0000	0.0000	0.0000
[2026-08-19 14:43:57]	[train]	52	0.9101	0.9500	1.0000	1.0000	1.0000
[2026-08-19 14:43:57]	[train]	53	0.9020	0.9600	0.8889	1.0000	0.8000
[2026-08-19 14:43:57]	[train]	54	0.9351	0.9571	0.9630	1.0000	0.9286
[2026-08-19 14:43:57]	[train]	55	0.8683	0.9250	0.8571	1.0000	0.7500
[2026-08-19 14:43:57]	[train]	56	0.5075	0.7257	0.6857	0.6857	0.6857
[2026-08-19 14:43:57]	[train]	57	0.7661	0.9167	0.8000	1.0000	0.6667
[2026-08-19 14:43:57]	[train]	58	0.7123	0.8286	0.8889	0.9231	0.8571
[2026-08-19 14:43:57]	[train]	59	0.6730	0.9667	0.6667	0.6667	0.6667
[2026-08-19 14:43:57]	[train]	60	0.5061	0.6692	0.5556	1.0000	0.3846
[2026-08-19 14:43:57]	[train]	61	1.0000	1.0000	1.0000	1.0000	1.0000
[2026-08-19 14:43:57]	[train]	62	0.6031	0.9500	0.6667	1.0000	0.5000
[2026-08-19 14:43:57]	[train]	63	0.8149	0.9667	0.8000	1.0000	0.6667
[2026-08-19 14:43:57]	[train]	64	0.3695	0.7000	0.0000	0.0000	0.0000
[2026-08-19 14:43:57]	[train]	65	0.7589	0.7875	0.8571	1.0000	0.7500
[2026-08-19 14:43:57]	[train]	67	0.6142	0.6625	0.8000	0.8571	0.7500
[2026-08-19 14:43:57]	[train]	68	0.5795	1.0000	0.5000	1.0000	0.3333
[2026-08-19 14:43:57]	[train]	69	0.5300	0.5400	0.5714	1.0000	0.4000
[2026-08-19 14:43:57]	[train]	71	0.6840	0.9000	0.7273	0.8000	0.6667
[2026-08-19 14:43:57]	[train]	72	0.5829	0.8200	0.7500	1.0000	0.6000
[2026-08-19 14:43:57]	[train]	73	0.4304	0.6000	0.6909	0.7308	0.6552
[2026-08-19 14:43:57]	[train]	74	0.5265	0.8333	0.0000	0.0000	0.0000
[2026-08-19 14:43:57]	[train]	75	0.7172	1.0000	0.6667	1.0000	0.5000
[2026-08-19 14:43:57]	[train]	76	0.0000	0.0000	0.0000	0.0000	0.0000
[2026-08-19 14:43:57]	[train]	77	0.3730	0.8048	0.0000	0.0000	0.0000
[2026-08-19 14:43:57]	[train]	79	0.8844	0.9200	1.0000	1.0000	1.0000

[2026-08-19 14:43:57] [train] Metric __rfdetr_effective_map__ improved by 0.016 >= min_delta = 0.001. New best score: 0.623

[2026-08-19 14:43:57] [train] [2026-08-19 14:43:57] [INFO] rf-detr - Best regular checkpoint saved to D:\Dev\模型\检测\2026-08-19_14_32_55\checkpoint_best_regular.pth (epoch 20, monitor=val/mAP_50_95, value=0.622912)

[2026-08-19 14:43:57] [train] [2026-08-19 14:43:57] [INFO] rf-detr - Best EMA mAP improved to 0.6121 (epoch 20)

日志窗口

